

IoT-Based Fertilizer Spiller Using ESP32 Microcontroller

Abstract — This paper presents the design and development of an Internet of Things (IoT)-based fertilizer spiller utilizing an ESP32 microcontroller. The system offers two operational modes: manual and automatic. In manual mode, the user sends commands using push buttons to control the direction of the fertilizer spiller. In automatic mode, a machine learning model trained on MPU6050 sensor data (accelerometer and gyroscope) recognizes specific gestures and direct movements. Previous research including studies by Zhang et al. (2019) and Kumar et al. (2018), validates the application of IoT in automated fertilizer distribution. They highlight the effects of increased efficiency and productivity with these applications. The experimental setup involved extensive data collection and training of a neural network model, achieving high accuracy in gesture recognition. Wireless communication between components was achieved through the ESP-NOW communication protocol with flexibility and reliable performance. This work reinforces the potential of IoT and machine learning to enhance agricultural processes with a prototype that can lead to more efficient and effective fertilizer application systems.

Keywords—Microcontroller, ESP32, IoT, fertilizer spiller machine learning, precision agriculture

I. INTRODUCTION

Moreover, we found that the use of IoT for automatically applying fertilizer has been explored by other studies. Our vision is grounded in a study presented by Zhang et al. (2019) who designed an automatic fertilizer applicator based on IoT technology using the Raspberry Pi platform [5]. Their system combined GPS and sensor data to apply fertilizers automatically, with accuracy, and promptly. This serves as proof that IoT devices are fit to help reduce labor costs and improve fertilizer use efficiency. Although their work relied on a microcontroller different from ours, the principles of using IoT for automated and precise fertilizer distribution are directly applicable to our proposed system.

Further, Kumar et al (2018) presented a smart agriculture based IoT solution, utilizing automated fertilizer spraying. Their system, controlled by an ESP8266 microcontroller, achieved substantial enhancements in fertilizer usage effectiveness and crop production [6]. This study presents a remarkable real-world case for the utilization of IoT-based systems in fertilizer administration, thereby substantiating the suitability and merits of the proposed ESP32-based fertilizer spiller.

To conclude, the reviewed literature reveals many growth and potential uses of the Internet of Things in agriculture. and it is now possible to control many aspects of agriculture continuously. The ESP32 microcontroller has evolved a lot, and it has been used in several applications to monitor our environments, like the driver `USE_ACCEL_2`, and a function to automatically control some things like light (control output), detecting and letting water the plants (light sensor), or even can be connected to a database and know when the cottage plants need its owner to feed then the person will get an alert by email.

The reports conducted served as a great source for a manual to guide the implementation of an IoT-based fertilizer spiller to be used with ESP32, which will surely help and contribute to the agricultural field to be more efficient and effective.

II. RELATED WORKS

Recently, a lot of progress has been made in the field of IoT-

based agricultural solutions. Several research have shown that there is great potential for using IoT in the area of precision farming which involves the utilization of IoT in agriculture. The following section is a literature review of selected works that helped us gain an understanding of the topic and prerequisites for an IoT-based fertilizer spiller using the ESP32 Microcontroller.

The IoT technology, for that part, has made them utilized in the supervision & controlling of agricultural causes and hence improving the efficiency and productivity of agriculture. For example, Kodali and Mandal (2016) established an automated smart irrigation system integrated with IoT and ESP8266 microcontroller which modulates water supply concerning soil moisture content proving the exploitation of the computerized agro-application [1]. In the same context, a precision agriculture framework based on Arduino was proposed by Jain and Bhatnagar (2016) for the watering and fertilization of a farm, with remarkable results in terms of resource optimization, and crop productivity [2].

Why ESP32 is one of the most popular devices for IoT use cases it comes with a dual-core processor, in-built Wi-Fi, and Bluetooth ready and has a very low power consumption. A huge number of smart agriculture applications have made use of this microcontroller. For example, Vasisht et al. Reference [3] implemented an IoT-based smart farming system with the combination of ESP32 to facilitate the monitoring of soil moisture, temperature, and touch or humidity to help farmers make rational irrigation and fertilization measurements. Their project demonstrates how the ESP32 can be used in versatile and efficient agricultural IoT applications.

Similarly, the integration of sensors with IoT devices has contributed to improved precision and reliability of agricultural monitoring systems. The study by Basit et al. In another work, Works [2018] presented an IoT soil monitoring system using an ESP32 microcontroller with multiple sensors to measure soil pH, soil moisture, and soil temperature. This system helped in sending real-time data to Farmers that will help them in properly using fertilizers and water [4]. The IoT-based fertilizer spiller in the example is designed to meet these same objectives and emphasizes the role of real-time monitoring in the optimization of agricultural inputs.

Moreover, we found that the use of IoT for automatically applying fertilizer has been explored by other studies. Our vision is grounded in a study presented by Zhang et al. (2019) who designed an automatic fertilizer applicator based on IoT technology using the Raspberry Pi platform [5]. Their system combined GPS and sensor data to apply fertilizers automatically, with accuracy, and promptly. This serves as proof that IoT devices are fit to help reduce labor costs and improve fertilizer use efficiency. Although their work relied on a microcontroller different from ours, the principles of using IoT for automated and precise fertilizer distribution are directly applicable to our proposed system.

Further, Kumar et al (2018) presented a smart agriculture based IoT solution, utilizing automated fertilizer spraying. Their system, controlled by an ESP8266 microcontroller, achieved substantial enhancements in fertilizer usage effectiveness and crop production [6]. This study presents a remarkable real-world case for the utilization of IoT-based systems in fertilizer administration, thereby substantiating the suitability and merits of the proposed ESP32-based fertilizer

spiller.

To conclude, the reviewed literature reveals many growth and potential uses of the Internet of Things in agriculture. and it is now possible to control many aspects of agriculture continuously. The ESP32 microcontroller has evolved a lot, and it has been used in several applications to monitor our environments, like the driver USE_ACCEL_2, and a function to automatically control some things like light (control output), detecting and letting water the plants (light sensor) or even can be connected in a database and know when the cottage plants needed its owner to feed then the person will get an alert by email.

The reports conducted served as a great source for a manual to guide the implementation of an IoT-based fertilizer spiller to be used with ESP32, which will surely help and contribute to the agricultural field to be more efficient and effective.

III. EXPERIMENTAL SETUP

The experimental setup consisted of two boards (manual and automatic) and the car (Fig. 1).



Fig. 1. Board Configurations Associated with Each Motion

The motor device consisted of plastic pieces to stabilize the electronic components onto the board, screws, two motors, two wheels, TB6612FNG Dual Motor Driver Carrier, and a battery. The software required to run the device was split into two, one for the manual mode being controlled with button presses, and another for the automatic mode, controlled by user gestures.

The hardware used for the manual and automatic mode controllers was composed of simple electronic components. For the manual mode, four buttons were connected to their respective pins on the ESP32 board. These pin configurations are provided in Table I.

The automatic mode setup included one button, an LED, an MPU6050 sensor to detect acceleration and gyroscope movement data, and an OLED display. To make this mode functional, we collected 200 data points for each motion by repeating the associated board configurations to each motion. A summary of the board configurations is provided in Fig. 2. Each motion had a duration of 1 second. For all movements, the board was initially held so the USB plug of the ESP32 board was facing in the direction of the user.

The front motion has the board tipped 90 degrees, so the backside of the board is showing to the user. The back motion is in the opposite direction, tipping 90 degrees so the board components are shown to the user. The left and right movements are self-explanatory, as they are tipped 90 degrees either left or right with the USB portal of the ESP32 facing towards the user. The spin motion involved spinning the entire board 360 degrees once. The stop motion required putting the

board down on the table from where it is held (approximately 15 centimeters from the table).

200 datasets were collected for each of these motions by pressing the button connected to the ESP32, which turned the LED on for a second. The associated board configurations were completed within that one-second duration of when the LED was turned on. The board always started from its initial position, which was having the board still and on the same side as shown in the spin and stop board configurations.

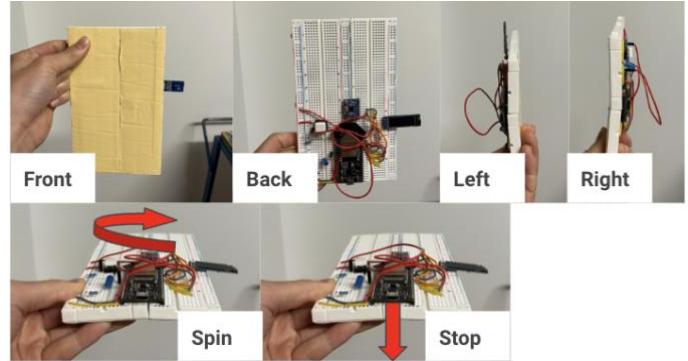


Fig. 2. Board Configurations Associated with Each Motion

TABLE II. PIN CONFIGURATION

ESP32	GPIO Pin	Function
#1 (Manual Mode Controller)	4	Button #4 (Front)
	17	Button #4 (Back)
	19	Button #4 (Left)
	32	Button #4 (Right)
#2 (Automatic Mode Controller)	35	Button #1
	27	LED #1 (blue)
	21	OLED display + MPU6050
	22	OLED display + MPU6050
#3 (Receiver)	17	AIN2
	18	AIN1
	21	BIN21
	22	BIN22
	23	PWMA
	16	PWMB
	19	STBY

The manual mode of this system consisted of four buttons, each controlling the forward, backward, left, and right motion of the car respectively. No button press was associated with stopping the system. A simple C++ code was written to assign each button to one motion and send commands via the ESP-NOW protocol.

Automatic Mode:

The automatic mode, on the other hand, leveraged a machine learning model to recognize specific gestures using data from an MPU6050 sensor (x,y,z values of accelerometer and gyroscope). Detected gestures, and similarly sent commands to the system using ESP-NOW. The first step of training the machine learning model included collecting 200 data points of six motions in total: forward, backward, left, right, spin, and stop. A simple circuit was created including an ESP32, LED, button, OLED display, MPU6050 sensor, and an SD card. For each button press, the LED turned on for a second, indicating it was collecting motion data for that duration. By repeating this process of data collection, all 1200 data points from 6 motions were compiled into their respective CSV file named based on the motion that was being collected. Some special libraries used were the SD.h to initiate I2C communication, send, and receive data over the I2C bus, SPI.h to enable allow for file operations such as opening, reading, writing, and closing files on an SD card, SPI.h to communicate with the SD card, and FS.h to allow manipulation of files and directories.

To prepare the collected motion data for further analysis and model training, each CSV file was preprocessed in Microsoft Excel. Each data column was appropriately labeled (eg. 'accel_x_1', 'accel_y_1', 'accel_z_1') and the last output column assigned a unique number to each gesture type across the rows. The file paths of each pre-processed CSV file were copied into the Python script to train a machine-learning model. This step involved using several libraries including Pandas and Numpy for data manipulation and numerical computations. Tensorflow and Keras handled the building and training of neural network models. Scikit-learn's train_test_split function divided the data into training and testing sets into 70/30 split. LabelBinarizer converted the output labels into a one-hot encoded format. Although the initial neural network was trained on this 70/30 split to evaluate performance, the model was then retrained using the full dataset to utilize all data for final testing. As a flow, the motion datasets were loaded, concatenated, and split according to their features and labels which were then used to train a neural network with two hidden labels within the TensorFlow library. The performance of this model was evaluated using a confusion matrix, and finally, converted into C++ code through the tf_porter library from Everywhere.ml. A cpp code was printed at the end of the script to copy into the motionModel2.h file which sets up compiler directives and macros to configure attributes and parameters for a machine learning mode. This entire step is summarised in Fig. 3.

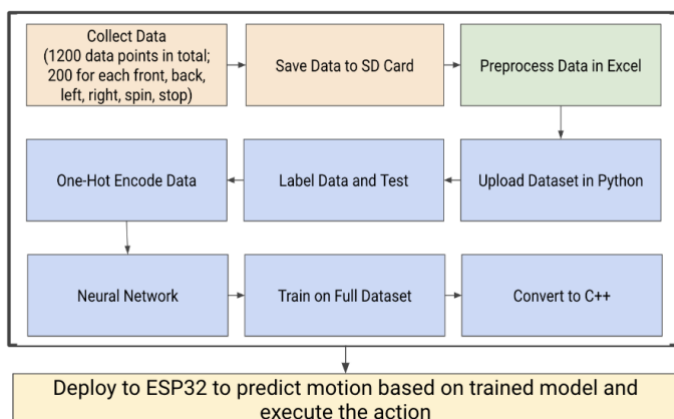


Fig. 3. Block Diagram of Automatic Mode

The pre-trained model was loaded onto the ESP32 board using the TensorFlow Lite for Microcontrollers library. Each of the 6 motions was tested 30 times to confirm the accuracy of the trained model. In the initial data collection, using only 100 data points for each motion, proved to be less accurate, proving the importance of having large data sets when training a machine learning model.

The model was collated into a code that set up communications with ESP-NOW for wireless data transmission and defined a struct 'MotionEvent' to represent detected motion types. This enabled motion gestures trained to send commands to the device.

To communicate between the ESP-32 microcontrollers we used the ESP-NOW communication protocol where two ESP-32 microcontrollers (connected to MPU6050 and 4 buttons) were masters and the third microcontroller (connected to the motor driver) was a slave as it received the signals from those two ESP-32 microcontrollers which then directed the motion of the motors.

Merits of ESP- NOW:

From the projects that we were assigned during the past 10 weeks, we realized that the demos that included communications depending on the wifi were only possible when we were not in our dorms. Using ESP-NOW communication was a good solution to this problem as it provided us the flexibility to work according to our schedules in a comfortable environment and continue to work on the project regardless of where we were present.

Apart from using ESP-NOW we also had an option of using Bluetooth however we avoided using Bluetooth communication since it lagged in some cases. More importantly, the code for implementing the Bluetooth communication seemed a lot more complicated which meant that there were more chances of running into various errors and debugging them which could have taken a lot more time than it did in the case where we implemented the ESP-NOW communication.

Cons of ESP-NOW:

One of the general cons of the ESP-NOW is that it does not allow communication over a larger distance like other communications like clouds and wifi which can be a drawback in real-life applications of our device where the fertilizers are to be spilled over a larger area. Furthermore, it is also prone to interference from other wireless devices operating in the same frequency band which affects the credibility and the reliability of this communication method.

V. RESULT AND DISCUSSION

Using the Python code explained in the implementation section, the confusion matrix shown in Fig. 4 was obtained. The rows represent the ground truth, and the columns indicate the predicted label. Values in each cell represent the number of data points. Fig. 4 is for the 70/30 testing, with most gestures being predicted correctly. Gestures 1 and 6, as indicated by the non-zero values of the diagonal in their corresponding rows, highlight some confusion. These non-zero values indicate instances where the model classified the feature incorrectly.

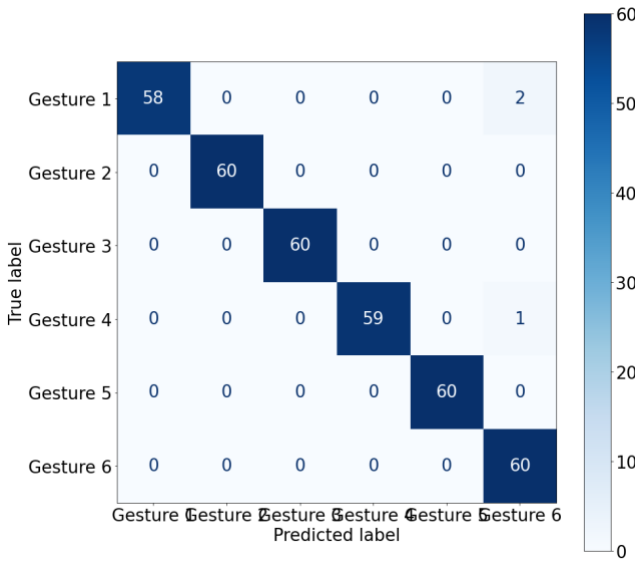


Fig. 4. Confusion Matrix of 70/30 Testing

The testing on the hardware performed very well, with no incorrect classifications (Fig. 5). All gestures were predicted as intended, shown below in the confusion matrix for this test trial before deploying to the system to control the car.

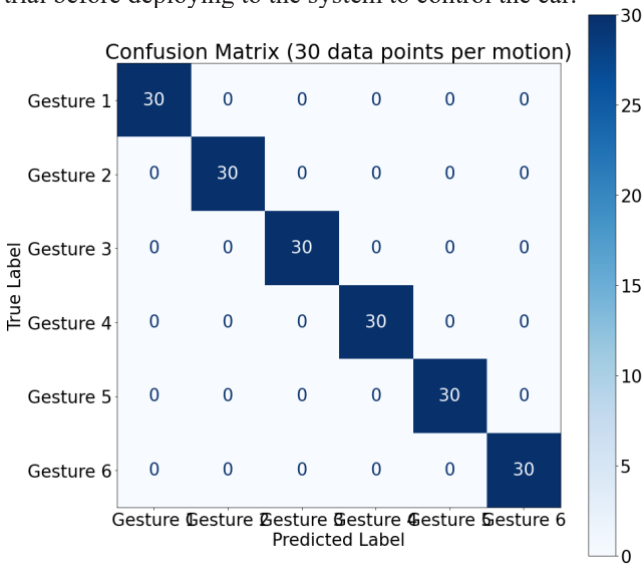


Fig. 5. Confusion Matrix of Pre-Trained ML Model Testing on Hardware

The system response latency was minimal, with the manual mode being less than a second with the push button commands and automatic mode being around a second after the LED turned off on the board, indicating the end of the motion data collected for that trial. The communication distance between the communicator and receiver edge nodes was around 220 meters.

V. CONCLUSION

This paper demonstrated the design and implementation of an IoT-based fertilizer spiller using an ESP32 microcontroller. The system offers two operational modes: manual and automatic. The manual mode uses push buttons to control the forward, backward, left, and right movements of the fertilizer spiller.

The automatic mode leverages a machine learning model trained on data collected from an MPU6050 sensor to recognize gestures associated with specific motions on the breadboard. The model achieved high accuracy in gesture recognition, as shown in the confusion matrix in the report.

The ESP-NOW communication protocol ensures wireless communication between the control unit and the motor driver. This approach offered greater flexibility compared to Wi-Fi-dependent communication protocols.

To leverage this IoT-based agricultural fertilizer spiller to solve pressing agricultural needs with unlimited time and

resources, enhancements could be made first, in the field readiness of this system. This includes ensuring the fertilizer spiller is robust enough to withstand outdoor conditions and can have a reliable power source such as solar panels to drive the motors and boards. Moreover, there would be a need to collect data for machine learning from multiple agricultural environments to meet the certain needs of the farmer and to improve its accuracy in gesture recognition and motion control. In terms of the communication, the current method using ESP-NOW communication protocol is limited in its communication distance. Instead, other methods such as LoRaWAN could be used in coordination with integrating the system with existing farm management systems.

Scalability and automation can be achieved by developing a central management system to handle multiple spillers using GPS and advanced navigation algorithms for a fully autonomous operation. Moreover, precision agriculture could be accomplished by integrating more sensors to the control car that could potentially take in real-time data and implement variable rate technology adjust the amount of fertilizer for certain sections of the field. And as is for all systems, there should be a user-friendly interface to supervise the machinery and the data being captured by the sensors.

Once the full system is analyzed, created, and tested, it would be essential to provide training programs and workshops to educate about the uses of this technology to farmers, especially since the agricultural field is resistant to change.

In conclusion, the project presented a prototype for automating fertilizer application in agricultural settings. The ESP32 microcontroller's capabilities and the effectiveness of the machine learning model contributed to a precise and efficient distribution system prototyping. Additionally, we have learned valuable lessons in terms of teamwork and communication which made this machine-learning integrated project possible with more ease. We have learned to utilize documentation and past work completed within the course to accomplish the ESP-NOW communication and troubleshoot pin issues with the motors and the manual mode controller. These experiences provided valuable lessons that would help us in our new positions and endeavors in the future.

By enhancing the current prototype and focusing on scalability, integration, and keeping the end-user demographics in mind, this system has potential to improve efficiency and productivity in agriculture.

While doing this project we learnt multiple lessons-technical and non-technical. Throughout the project, we gained invaluable knowledge and skills that will serve us well in our future professional endeavors. One of the most critical lessons was understanding how to train and deploy machine learning models onto hardware. We learned the importance of high-quality data and preprocessing techniques to ensure robust model performance. This included data normalization, scaling, and handling missing values, as well as the significance of splitting data into training, validation, and test sets to prevent overfitting. Additionally, we explored hyperparameter tuning to optimize model accuracy and performance. When it came to deployment, we delved into converting trained models into formats suitable for embedded systems or microcontrollers and integrating these models with hardware components to ensure efficient use of computational resources and minimal latency. This comprehensive process provided a solid foundation in both theoretical and practical aspects of machine learning deployment.

Another significant lesson was the effectiveness of the divide-and-conquer strategy in managing complex projects. By breaking down the project into manageable tasks and distributing them among team members, we made the overall process more efficient and achievable. Clearly defining roles and responsibilities allowed each team

member to focus on specific aspects of the project, leveraging their strengths and expertise. Regular progress meetings ensured alignment with the overall project goals, and project management tools like Gantt charts and Kanban boards helped visualize the project timeline and progress.

Effective communication emerged as a vital component in overcoming challenges and ensuring project success. Articulating problems clearly and collaboratively finding solutions was critical. Encouraging team members to openly share the issues they encountered facilitated timely identification of potential roadblocks, and documenting these problems helped in prioritizing and addressing them effectively. Brainstorming sessions allowed us to collectively explore solutions, leveraging diverse perspectives within the team, and implementing a feedback loop ensured continuous improvement and adaptation. This transparent and supportive team environment turned problems into opportunities for collective growth and learning.

Additionally, the ability to efficiently navigate and utilize online documentation proved crucial. Given the rapidly evolving nature of technology, the skill to independently learn and adapt to new tools and commands is invaluable. Regularly consulting official documentation, forums, and tutorials provided up-to-date information and best practices. Applying this information to our specific project needs enhanced our problem-solving skills and prepared us for professional scenarios where we will frequently encounter new technologies and challenges. These lessons, encompassing technical skills, effective project management, and collaborative problem-solving, have equipped us with valuable tools and insights for our future professional endeavors.

REFERENCES

- [1] Kodali, R. K., & Mandal, S. (2016). IoT-based weather and crop monitoring system. *2016 IEEE 6th International Conference on Advanced Computing (IACC)*, 19-26.
- [2] Jain, S., & Bhatnagar, V. (2016). Precision agriculture using Arduino and wireless sensor network. *2016 IEEE 3rd World Forum on Internet of Things (WF-IoT)*, 509-514.
- [3] Vasisht, P., Lim, K. J., & Sharma, A. (2017). Smart farming using IoT and ESP32. *2017 International Conference on Computational Intelligence in Data Science (ICCIDS)*, 1-5.
- [4] Basit, A., Qamar, U., & Lee, S. (2018). IoT-based soil monitoring and irrigation control system. *2018 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, 1-6.
- [5] Zhang, H., Wang, X., & Li, Y. (2019). Design and implementation of an IoT-based automatic fertilizer applicator. *2019 IEEE 5th World Forum on Internet of Things (WF-IoT)*, 649-653.
- [6] Kumar, N., Kaushal, R., & Singh, S. (2018). Smart agriculture: An IoT-based approach. *2018 IEEE International Conference on Advances in Computing, Communications, and Informatics (ICACCI)*, 693-698.

APPENDIX

Python, C++ Codes, and Datasets with Descriptions: [ECE-328 Final project](#)